Министерство науки и высшего образования РФ
ФГАОУ ВПО
Национальный исследовательский технологический университет «МИСиС»

Институт Информационных технологий и компьютерных наук (ИТКН)

Кафедра Инфокоммуникационных технологий (ИКТ)

Отчет по практической работе
по дисциплине «Методология разработки ПО»
на тему «Разработка ПО для заказчика по требованиям»

Выполнил:
студент группы БИВТ-23-СП-1

Широковских И.С.

Москва, 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **3**
РАЗРАБОТКА ФОРМ И КЛАССОВ **6**
1. Форма авторизации. 6
2. Форма товаров. 7
3. Форма добавления товара. 9
4. Форма редактирования товара. 10
5. Форма модального окна для товара. 10
6. Форма заказов. 10
7. Форма добавления заказа. 11
8. Форма редактирования заказа. 11
9. Форма модального окна для заказа. 12

Настройки доступа. 12
ЗАКЛЮЧЕНИЕ **13**
ПРИЛОЖЕНИЕ **14**

ВВЕДЕНИЕ

В данной работе я опишу выполненную практическую работу, всё, что было разработано и, как результат, саму написанную программу.

Весь код был написан на языке программирования C#, в качестве среды разработки была выбрана Visual Studio v2022 C# вида Windows Forms Framework 4.7.2, поскольку поставленные требования больше подходят именно для такого типа приложения. Также СУБД была выбрана MS SQL Server v2021 с хорошим API и лёгкой связью с ПО.

Вначале будут описаны все классы и формы с ссылкой на изображения в приложении (в конце), по ходу так же будут описаны основные методы и инструменты для разработки данного ПО.

Данный отчёт будет являться визуальным помощником в понимании приложенной схемы по ГОСТ 19.701-90 с визуальным тестированием и облика ПО.

Все файлы и изображения хранятся внутри проекта в папке /bin/Debug, откуда и берутся, и кладутся все изображения.

БАЗА ДАННЫХ И СВЯЗЬ С ПРИЛОЖЕНИЕМ

Для начала была разработана по приложению №2 (архив с файлами) по таблицам база данных с четырьмя сущностями, представленные в виде таблиц с данными по аналогии Excel файлов из приложения, которые имел заказчик.

В базу данных вошли **4 таблицы:** Товары (Products), Заказы (Orders), Пункты выдачи (Points) и Пользователи (Users), где несколько соответствующих атрибутов и связями по внешнему и первичному ключам между таблицами. Полную схему БД с атрибутами, типами данных и связями можно посмотреть ниже на *рисунке 1*. Данная схема была сделана на сайте *Drawsql.app*.

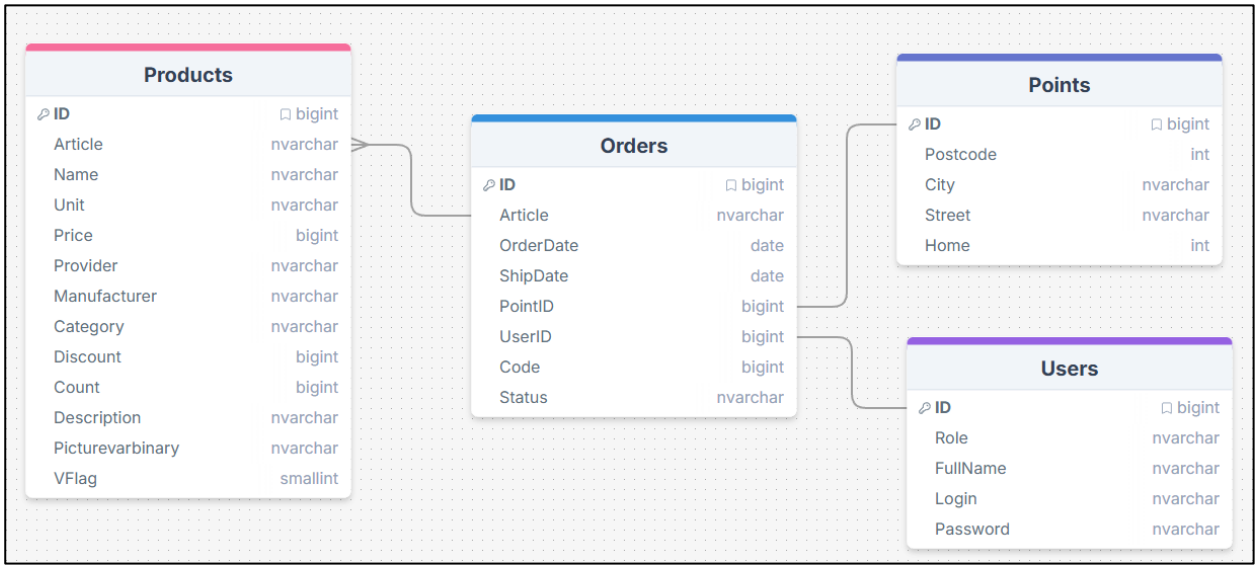

Рис. 1 – «Схема базы данных»

После разработки схемы, началась разработка БД в MS SQL Server. С помощью API этой СУБД было легко и быстро сделать необходимые таблицы и заполнить их предоставленными данными. Итоговая база данных стала называться “ShoesShop”.

После создания БД можно приступать к разработке ПО. Самое важное вначале сделать правильную бесперебойную связь БД. Для этого был создан класс “DataBase”, в котором была функция создания подключения и успешного планового завершения связи, чтобы не перегружать БД, с помощью встроенного класса SQLConnection. В этот класс также вошли две публичные статические функции класса, чтобы к ним можно было откуда угодно обращаться:

- ***GetDataFromDB(string query)*** – данная функция принимает на вход строку SQL-запроса к базе данных, открывает подключение и передаёт её в СУБД, а та в свою очередь возвращает таблицу, полученную из запроса, которую и возвращает функция. Данная функция умеет только читать данные по SQL-запросу, но не модифицирует их.

- ***EditDataFromDB(string query)*** – данная функция работает аналогично функции выше, но она не читает данные, а только редактирует и удаляет записи из БД посредством SQL-запроса.

Само подключение будет локальным (так как не сказано делать его общим), поэтому используется только текущий хост и имя локального сервера.

А также были обработаны возможные исключения, пользователь с ними не встретится, так как это при разработке было нужно, но было необходимым для контроля подключения при раззработке.

Общий вид данного класса с подключением, который будет использоваться повсеместно представлен ниже на *рисунке 2*.

```
Ссылок: 61
public class DataBase
{
    Ссылок: 2
    public DataBase()
    {

    }
    static string serverName = "(localdb)\\MSSQLLocalDB";
    static string dataBaseName = "Shoes Shop";

    SqlConnection con = new SqlConnection($@"Data Source={serverName};Initial Catalog={dataBaseName};Integrated Security=True;");
    Ссылок: 1
    public void OpenCon()
    {
        if (con.State == System.Data.ConnectionState.Closed)
        {
            con.Open();
        }
    }
    Ссылок: 2
    public void CloseCon()
    {
        if (con.State == System.Data.ConnectionState.Open)
        {
            con.Close();
        }
    }
    Ссылок: 2
    public SqlConnection GetConnection()
    {
        return con;
    }

    // Функция получения данных из SQL-запроса к БД
    Ссылок: 46
    public static DataTable GetDataFromDB(string query)
    {
        DataBase db = new DataBase();
        SqlDataAdapter adapter = new SqlDataAdapter();
        DataTable dt = new DataTable();
        SqlCommand cmd = new SqlCommand(query, db.GetConnection());
        adapter.SelectCommand = cmd;
        adapter.Fill(dt);
        db.CloseCon();

        return dt;
    }


    // Функция выполнения команды из SQL-запроса в БД
    Ссылок: 10
    public static void EditDataFromDB(string query)
    {
        try
        {
            DataBase db = new DataBase();
            SqlCommand cmd = new SqlCommand(query, db.GetConnection());
            db.OpenCon();
            cmd.ExecuteNonQuery();
            MessageBox.Show("Изменения приняты!", "Успех!");
            db.CloseCon();
        }
        catch (Exception ex)
        {
            MessageBox.Show(ex.Message);
        }
    }
}
```

Рис. 2 – «Код класса DataBase с подключением к БД»

РАЗРАБОТКА ФОРМ И КЛАССОВ

Следующим важным этапом после соединения ПО с созданной БД будет непосредственная разработка самого ПО со всеми требованиями. Так как приложение на базе Windows Forms, то в нем будет 9 форм со своими назначениями, которые будут описаны ниже.

1. **Форма авторизации.**

Данная форма предназначена для авторизации в системе пользователя, чтобы прошедший эту форму человек имел заранее определённый доступ к ПО, соответствующий его роли. Также предусмотрен гостевой режим, который предоставляет ограниченный доступ к ПО и пропускает форму авторизации без введения данных.

В данной форме по соображениям удобства и структурированности было принято решение использовать паттерн проектирования «Chain Responsibility», который в случае неуспеха передает введенные данные дальше по цепочке иерархии пользователей. Для этого были созданы соответствующие классы для каждой роли: Admins, Managers, Clients и Guests, которые наследуются от общего интерфейса Users, который задает общие правила для всех пользователей, такие как функция, которая применяет к объекту класса объект нижестоящего класса, и функция проверки существование пользователя с такими данными и его роли. В случае неуспеха передаёт эти данные, которые хранятся во временном объекте, созданном с такими же параметрами, классу ниже, и так далее, пока не дойдет до нижнего класса, где в случае неуспеха выведется сообщение с отсутствием такого пользователя. В случае же успеха, то тот класс, в котором прошла проверка, запускает соответствующий таймер на 3 секунды (чтобы успеть прочитать пользователю сообщение об успехе авторизации), который в следующую открывающуюся форму списка товара передаёт ID и роль вошедшего.

Сама диаграмма классов вложена в проект и выглядит так, как показано на *рисунке 2.2* в приложении ниже.

Внешний вид формы и при успешной регистрации можно посмотреть в приложении *(рисунки 3 и 4)*.

2. **Форма товаров.**

На этой форме расположен основной функционал программы для пользователей. В переданной информации от формы авторизации, заполняются поля с именем авторизованного пользователя и его ролью. В зависимости от роли на данной форме некоторые элементы ограничены в использовании.

В виде элемента **dataGridView** расположено стилизованная красиво таблица с данными из БД, так как в базе на строках хранятся данные в сыром виде, а нам необходимый формат записей в таблице внутри ПО, то необходимо пересмотреть таблицу с помощью специально созданной функции ***FillTable(DataTable dt),*** которая принимает эту сырую таблицу (она может быть отсортирована SQL-запросами или полная, или отфильтрована и тп) и заполняет таблицу внутри ПО в нужном и красивом формате.

Картинки здесь берутся по имени из БД и по полному пути, включая имя, из папки описанной выше, но, чтобы в дальнейшем была возможность удалять, и не занимать память в оперативной памяти, было принято решение написать функцию ***CloneImg(string path),*** принимающая полный путь оригинала и выдает клонированное изображение (не копированное, а клонированное!), которое не занимает место и оригинал впоследствии может быть удален, также работают обработчики событий, особенно если по данному пути ничего не найдено.

В требованиях были указаны сортировка по количеству на складе, фильтрация по поставщику и поиск по всем текстовым полям.

Чтобы выполнить сортировку было решено применить переключатель в виде **pictureBox**, который если не активен отображает крест, а если идет фильтрация, то стрелку вверх или вниз в зависимости от вида сортировки (возрастание и убывание соответственно). Вид сортировки переключается одним кликом по картинке, а чтобы ее отключить нужно дважды кликнуть туда.

Поиск в виде строки поиска с текстом заглушки, который автоматически убирается при активации строки поиска и возвращается при деактивации (если пустая строка). Для этого были использованы соответствующие обработчики событий этого элемента.

Фильтрация оформлена в виде **comboBox**, уникальные элементы которого автоматически подгружаются из БД при запуске формы с помощью соответствующего SQL-запроса.

Теперь необходимо было сделать одновременную динамическую фильтрацию всего, чтобы не нужно было нажимать лишние кнопки. Я решил для каждого из этих элементов сделать соответствующее поле, в котором будет храниться текущий выбранный отправитель: у поиска - текст; у фильтрации - выбранный индекс; у сортировки - выбранное состояние (от 0 до 2), а в соответствующих обработчиках событий (для поиска - изменение текста; для фильтрации - изменение выбранного элемента; для сортировки - нажатие на кнопку один раз (двойной клик сбрасывает состояние до 0 и меняет картинку на крест)) эти поля соответственно меняются на выбранное в текущий момент времени. Используя все вышеперечисленное, я написал

функцию ***Mix()***, которая удаляет предыдущие записи из таблицы и строит по кусочкам из этих трёх полей с текущим состоянием и положением элементов этих фильтраций единый SQL-запрос для получения соответствующей таблицы (в качестве заглушки для фильтрации в блоке WHERE применяется всегда истинное выражение, чтобы WHERE была бессмысленной), таким образом пересобирается новая таблица отсортированная. И чтобы все запросы выполнялись динамически, эта функция была прописана во все обработчики событий в конце для этих элементов.

Все фильтры можно сбросить одной кнопкой, предусмотренной под строкой поиска, которая не только возвращает все поля к первоначальному виду, но и обнуляет поля и возвращает исходную таблицу товаров без фильтрации.

Также на этой форме есть кнопка «Выйти» для возвращения на предыдущую форму авторизации, закрыв текущую форму и показав ту старую.

Были добавлены ещё кнопки для добавления товара, которая открывает новую форму добавления товара, передавая текущую форму (чтобы заново её показать и все сохранилось), а текущая форма скрывается (но не закрывается), и кнопка для перехода на страницу заказов, открывая новую форму заказов с передачей туда этой формы, чтобы можно было вернуться назад и все сохранилось, и ID пользователя, который нажал на эту кнопку.

А чтобы редактировать товар, необходимо дважды нажать по любой ячейке в выводимой таблице и откроется новая форма редактирования товара, которая передает текущую форму и выбранный товар по индексу выбранной ячейки.

Общий вид формы при различном использовании показан на *рисунках 5 – 8* в приложении ниже.

3. **Форма добавления товара.**

Данная форма открывается при нажатии на кнопку «Добавить» на предыдущей форме, форме товаров, которая делает форму товаров недоступной, но видимой, и передаёт эту недоступную форму в конструктор этой формы. Внутри неё разместились поля **textbox** (название, поставщик, артикул, цена, скидка, единица измерения, кол-во на складе, описание) для соответствующих требованиям атрибутов и **combobox** (категория и производитель). У выпадающих списков соответствующие уникальные элементы берутся из БД и заполняются в качестве элементов этого списка, которые можно выбрать с помощью определённого SQL-запроса.

А также **pictureBox** с изображением, которое можно загрузить. При двойном нажатии на pictureBox открывает диалоговое окно открытия файлов, при выборе файла сразу проверяется расширение файла, принимаются только PNG, JPEG, JPG. Если расширения не такие, то выводятся модальное окно с объяснением какие файлы принимаются. Далее, если файла с таким именем нет в папке приложения с фотографиями, то файл копируется туда и ему задается случайное имя из 10 символов английского алфавита, это сделано для того, чтобы те файлы, которые уже есть в папке приложения и записаны в БД не конфликтовали по имени с новыми файлами, которые хотят загрузить туда, у файлов может быть одинаковое имя, но они сами разные, и чтобы все равно можно было загрузить туда фото, но с другим именем.

Все поля textbox проходят проверку по заполненности (чтобы не были пустыми, так как в БД не разрешены NULL), а также числовые поля (цена, кол-во на складе, скидка) проходят дополнительную проверку на числовое значение, у скидки и кол-во — это целые числа, а у цены десятичные. Для этого написаны специальные функции проверки строковых textbox и числовых, и они вызываются по событию изменения содержимого в textbox. Если какая-то проверка не проходит, то кнопка «Создать» исчезает и не доступна, и выводится сообщение с тем, что конкретно не так. И также для каждого поля своя переменная, которая также перезаписывается в событии при прохождении проверки.

В comboBox проверку можно пройти, если выбрать хотя бы какой-то элемент, кроме начальной строки.

Если все поля заполнены корректно, и кнопка не исчезла, то при ее нажатии собирается SQL-запрос на вставку нового элемента в таблицу, где значения атрибутов такие, как соответствующие созданные поля для этих атрибутов. И этот запрос выполняется той самой функцией EditDataFromDB(string query), и новая строка появляется в таблице.

Внешний вид и функционал можно посмотреть на *рисунках 9 – 11* в приложении ниже.

4. **Форма редактирования товара.**

Данная форма создана по аналогии с предыдущей формой, но теперь в нее передается через конструктора формы еще и ID редактируемого товара, чтобы при загрузке формы все эти поля заполнялась данными такими, как в БД. А также загружалась картинка в pictureBox из папки приложения по имени из БД.

А также кнопка удаления товара, которая открывает новую форму модального окна для товара, с предупреждением и подтверждением, которой передаётся текущая форма и ID товара, чтобы именно она удаляла товар.

А весь остальной функционал и внешний вид идентичен форме добавления товара со всеми проверками и изменениями данных в БД.

Внешний вид и функционал можно посмотреть на *рисунках 12 – 13* в приложении ниже.

5. **Форма модального окна для товара.**

Эта форма открывается из предыдущей формы редактирования товара и получает ID редактируемого товара. На ней две кнопки «Отмена» и «ОК», при нажатии на отмену или закрытии формы, данная форма закрывается и снова доступна предыдущая форма, как будто и ничего не открывалось. А при нажатии на «ОК» идёт проверка наличия этого товара в незавершенных заказах, так как в БД можно найти товар по артикулу заказа, там через делиметер расположены товары в заказе и за ними их количество, и путем рассплитивания данных с последующей проверкой каждого слова в артикуле заказа, мы проверяем все заказы, и просматриваем артикул товара на совпадение там, если есть хотя бы одно совпадение, то выводим сообщение о невозможности удаления, и закрываем форму, как при нажатии на кнопку отмены, а если нет, то товар безвозвратно удаляется из таблицы путем той же функции EditDataFromDB(string query) с соответствующим SQL-запросом.

Внешний вид и функционал можно посмотреть на *рисунках 14 – 15* в приложении ниже.

6. **Форма заказов.**

Данная форма включает в себя встроенную стилизованную под требования таблицу заказов из БД, аналогичную с формы товаров, но без изображений и сортировок, фильтров и поиска, что упрощает разработку. Вся технология данной и последующих форм аналогичные их соответствующим копиям форм товаров, но проще, за счёт отсутствия множество атрибутов и изображения. Так же при двойном нажатии на ячейку открывается форма редактирования заказа, и имеется кнопка добавления заказа, которая открывает форму добавления заказа и передаёт текущую форму заказа, делая ее недоступной для редактирования, чтобы не открывать множество окон добавления и редактирования заказов. Если закрыть текущую форму, то откроется так же сохраненная форма товаров, которая была скрыта.

Внешний вид и функционал можно посмотреть на *рисунке 16* в приложении ниже.

7. **Форма добавления заказа.**

Эта форма аналогична по внешнему виду и коду форме добавления товара, но из-за специфических атрибутов у заказов, здесь добавлены дополнительные поля для дат, а именно для дня, месяца и года, эти поля должны быть целочисленные, и их проверка осуществляется путём пробы конвертировать введенные три переменные, обозначающие день, месяц и год, в объект класса DateTime, который обозначает существующую дату, и если даже введены целые числа, то проверка может не пройти при несуществовании такой даты, например, 30.02.

Данная функция также динамически проверяет нужные поля на заполненность, поэтому в событие textbox при изменении текста вызывается эта функция и динамически проверяет правильность введенной даты. Если всё правильно, то сообщение о соответствующей ошибки исчезает снизу, и правее вводимых данных покажется та дата, которая была введена. И так же для каждой даты есть соответствующее поле на форме, чтобы хранить текущую выбранную дату.

И ещё одно интересное добавление, что в адрес пункта выдачи, элементы которого представлены в выпадающем списке, можно добавить свой персональный адрес, то есть помимо выбора элемента в само поле combobox можно ввести своё значение, которое так же проходит проверку. Заполнение должно происходить по шаблону, используемое в элементах ниже, то есть через запятую четыре значения должны быть представлены, и при этом первое и последнее значение – это целые числа, это также проверяется, так как для каждого значения из этих свой атрибут с соответствующим типом данных, и в таблице адресов пункта выдачи всего четыре атрибута.

И в случае, если всё введено верно, то появляется кнопка создания заказа, которая также формирует SQL-запрос из введенных значений и проверяет персональный адрес или выбранный из списка. После чего добавляет заказ и закрывает форму, активируя предыдущую форму заказов.

Внешний вид и функционал можно посмотреть на *рисунках 17 - 19* в приложении ниже.

8. **Форма редактирования заказа.**

Эта форма идентична предыдущей и аналогична форме редактирования заказа, но здесь так же добавляется кнопка удаления заказа, открывающая форму модального окна для заказа и передающая туда ID выбранного заказа. И при загрузке формы все поля на форме подгружаются для данного товара из БД автоматически. Все остальные проверки и схемы такие же, как и у добавления заказа.

И в конце идёт не добавление заказа, а изменение с соответствующим SQL-запросом.

Внешний вид и функционал можно посмотреть на *рисунках 20 - 22* в приложении ниже.

9. **Форма модального окна для заказа.**

Эта форма оформлена так же, как и форма модального окна для товара, но теперь, нажав на кнопку подтверждения, удаления производится без дополнительных проверок (нечего проверять), и форма закрывается.

Внешний вид и функционал можно посмотреть на *рисунке 23* в приложении ниже.

Настройки доступа.

После создания и написания кода для форм, всё, что остаётся, это для соответствующих ролей ограничить функционал форм, для этого были написаны 3 функции для каждой роли, кроме администратора, поскольку изначально все элементы форм включены, то администратор имеет полный доступ к ним, и его не надо ограничивать.

Для Менеджеров ограничены кнопки добавления товара и заказа, а также событие двойного нажатия на ячейку на таблице заказов и товаров, чтобы нельзя было их редактировать, из-за чего формы добавления товара, добавления заказа, редактирования товара, редактирование заказа теперь стали им недоступны, так как их неоткуда им открыть.

Для Авторизованных клиентов, помимо ограничений, наложенных менеджерам, добавляется сокрытие фильтра, поиска и сортировки, они были отключены, и убрана видимость с них, из-за чего они теперь не могут изменять таблицу товаров, но также могут просматривать заказы (по требованиям так).

А в гостевом режиме, помимо всех описанных выше ограничений, убрана кнопка перехода на страницу заказов, что уже ограничивает дополнительно доступ к форме заказов. То есть, по сути, остаётся только список товаров, который можно листать и просматривать.

Структуру всех форм и их связь, и доступ к ним можно посмотреть в дополнительном pdf файле.

ЗАКЛЮЧЕНИЕ

В результате было разработано полноценное программное обеспечение по всем приложенным требованиям со связью базы данных, спроектированной специально по требования и данные заказчика, а также наглядно показано и описано всё, что происходит в коде и на формах приложения.

Ниже можно посмотреть визуализацию всего сказанного в приложении.

ПРИЛОЖЕНИЕ

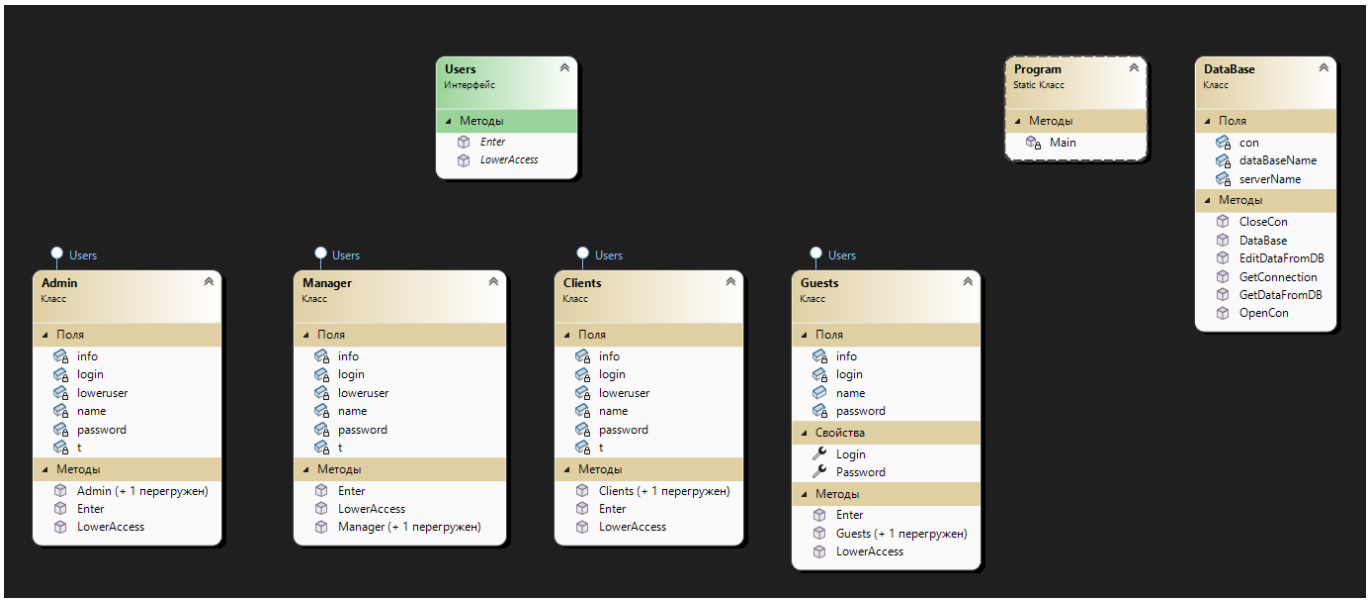

Рис. 2.2 – «Диаграмма классов в ПО»

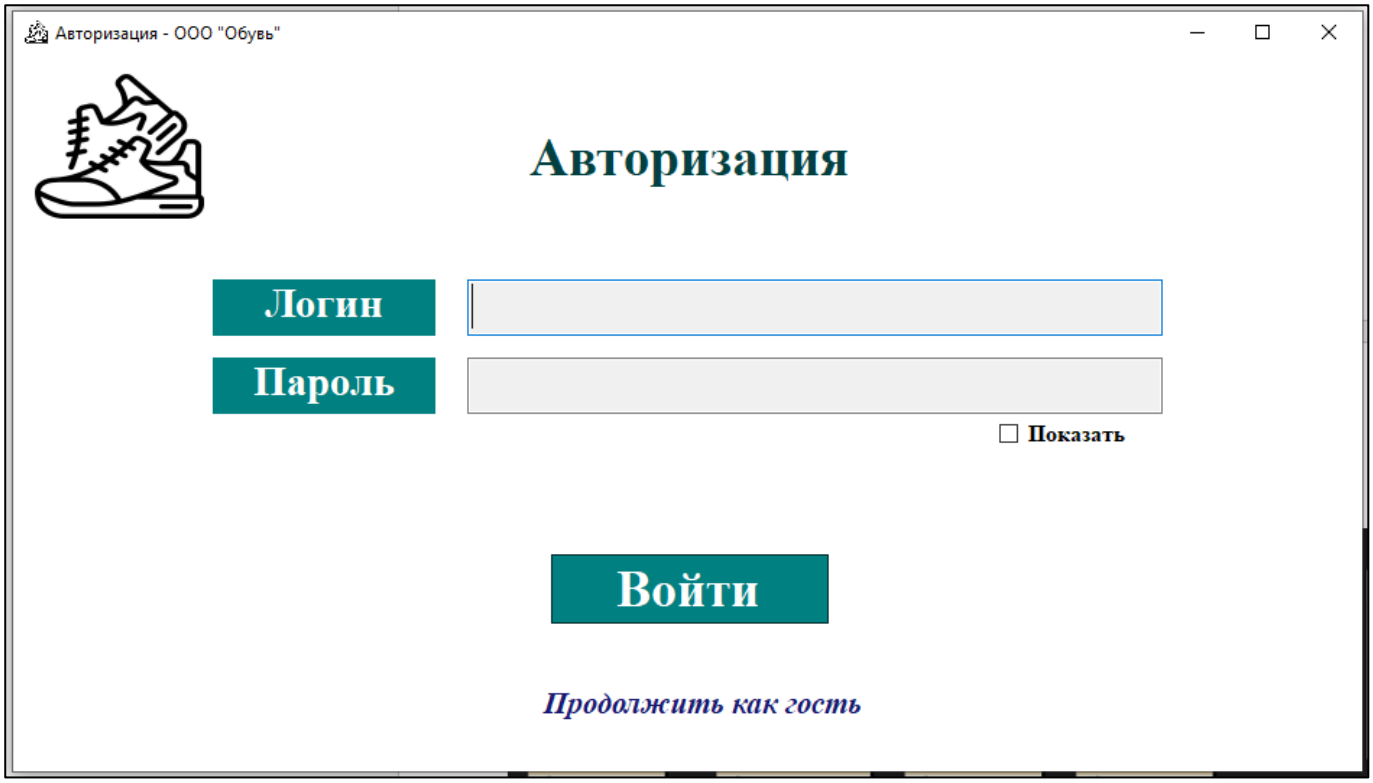

Рис. 3 – «Форма авторизации: общий вид»

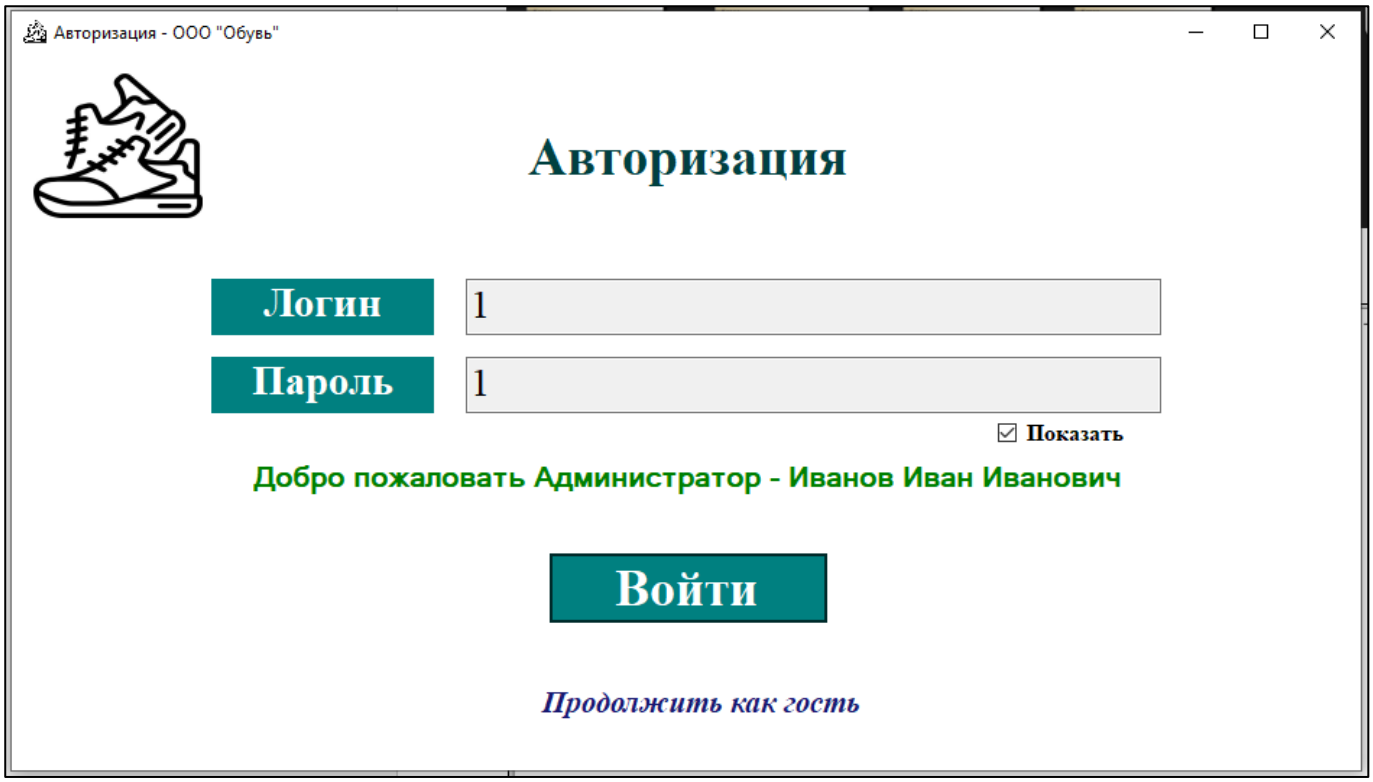

Рис. 4 – «Форма авторизации: успешный вход»

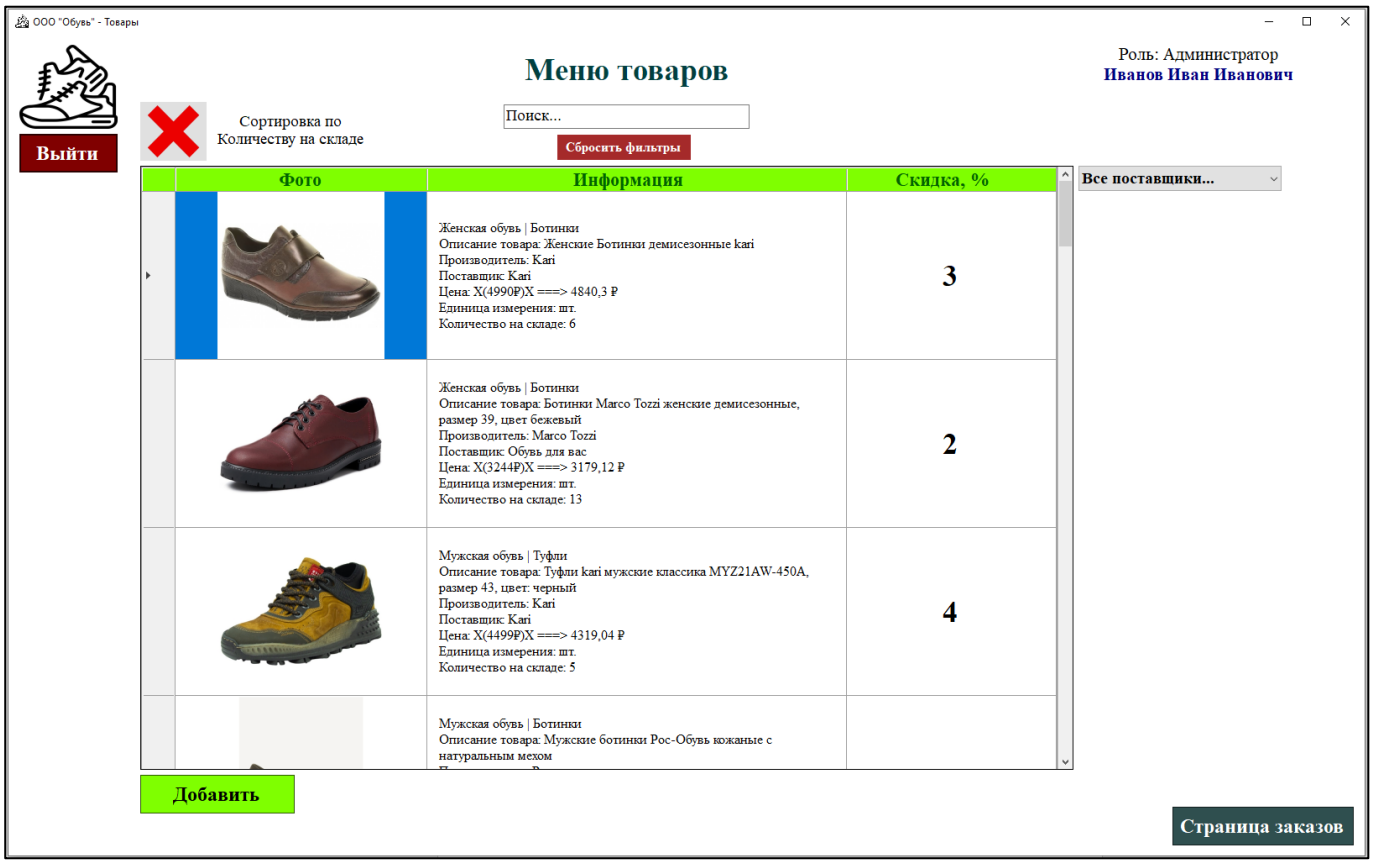

Рис. 5 – «Форма товаров: администратор»

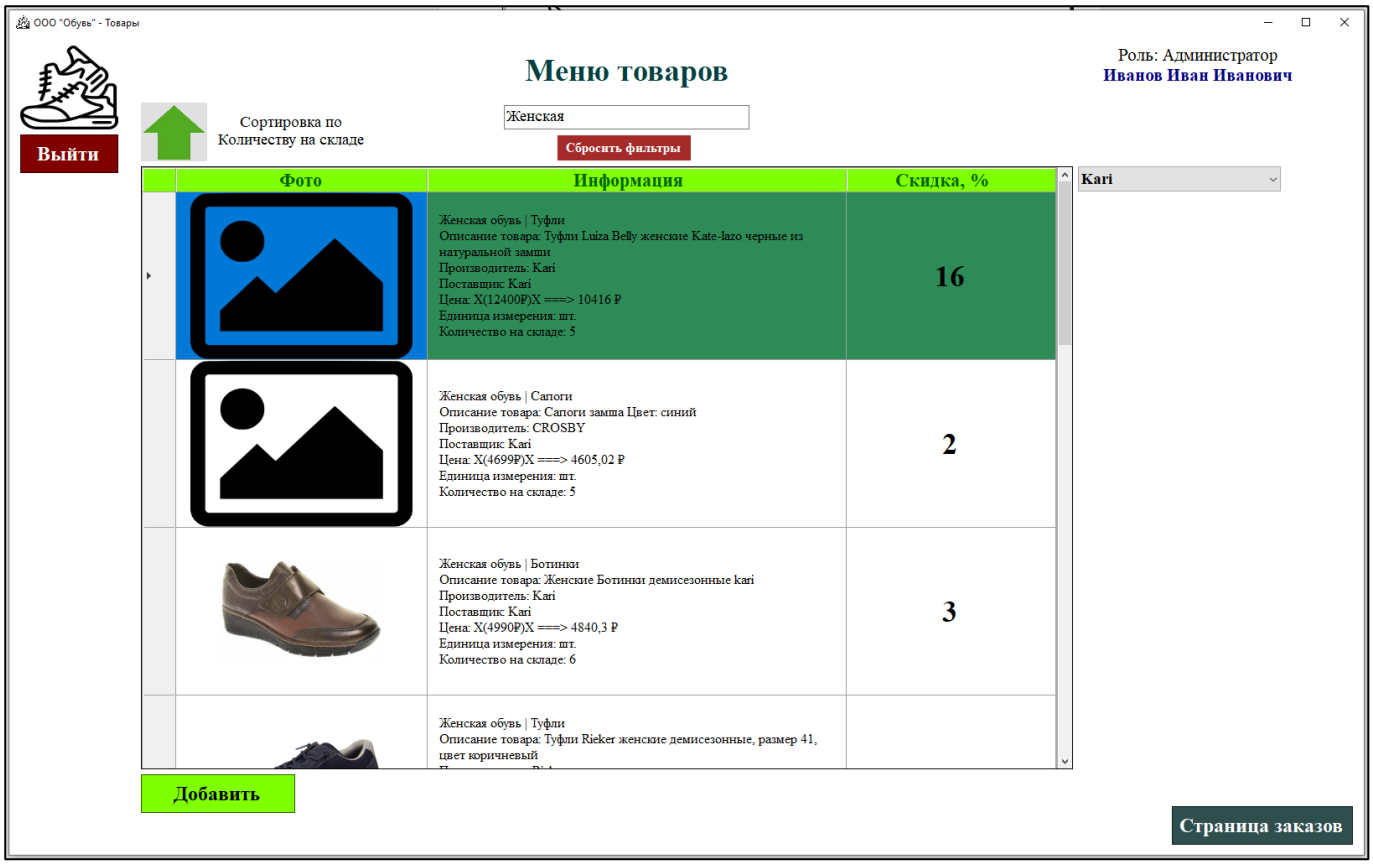

Рис. 6 – «Форма товаров: администратор – применение всех фильтров»

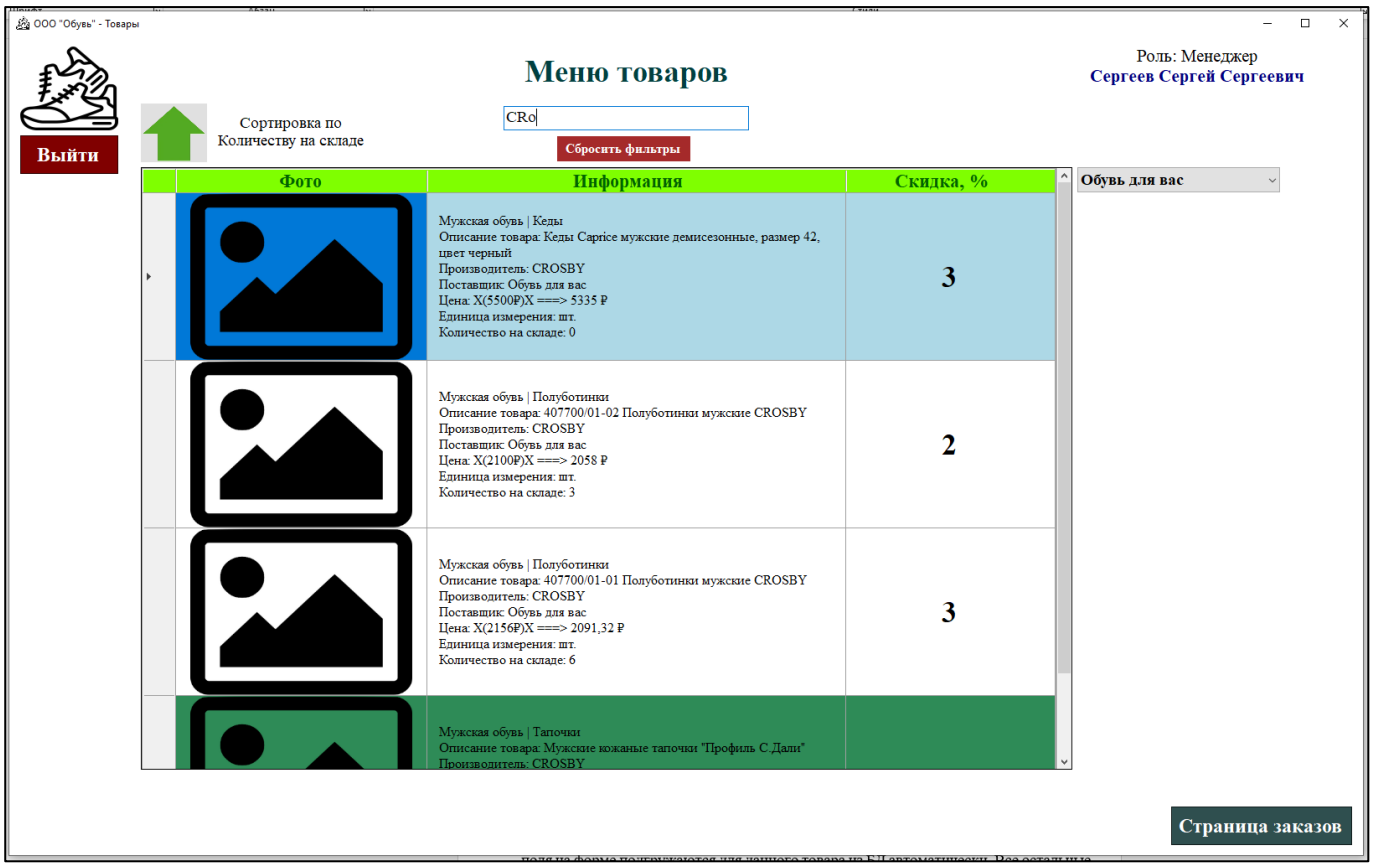

Рис. 7 – «Форма товаров: менеджер – применение других фильтров»

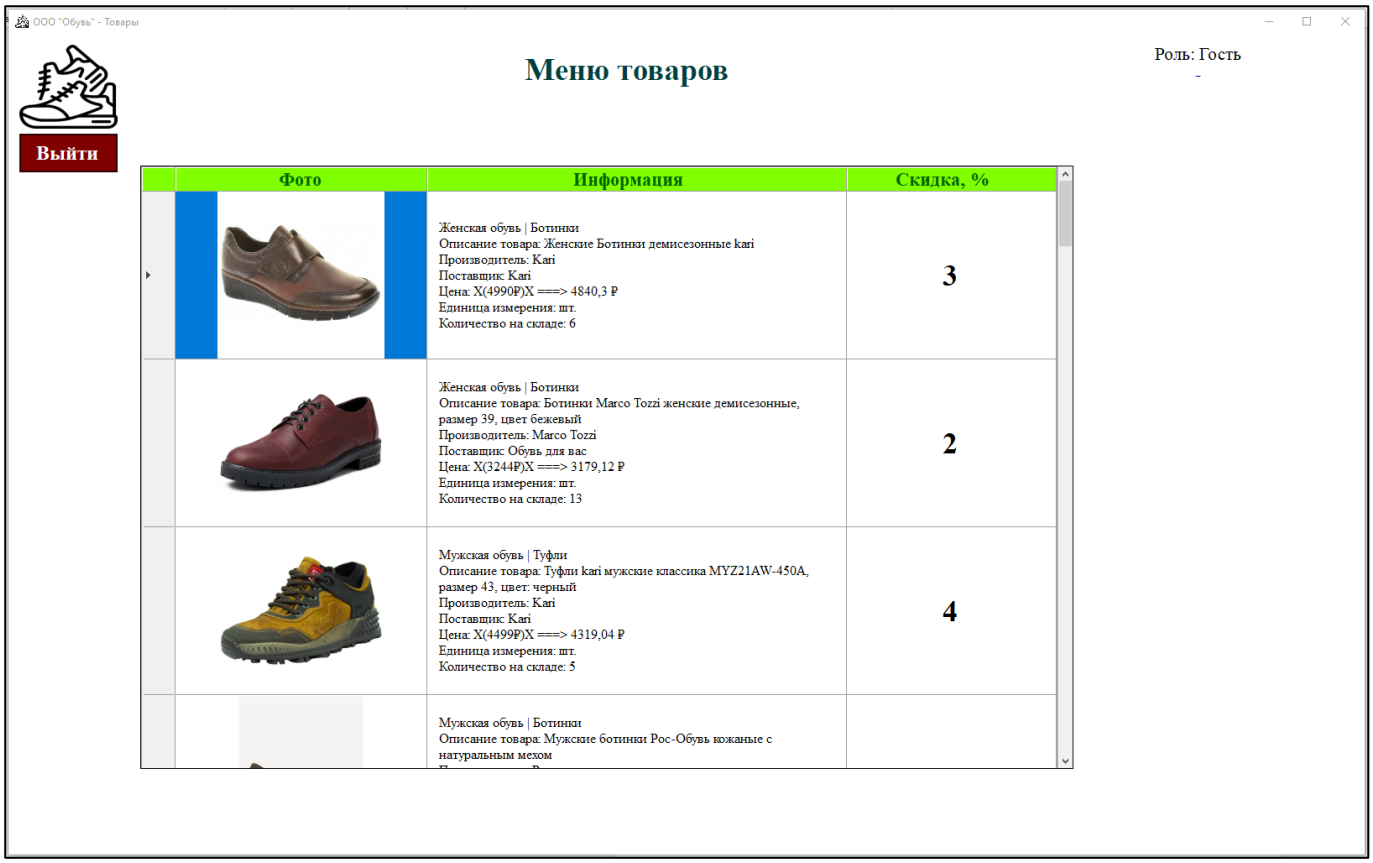

Рис. 8 – «Форма товаров: гостевой режим»

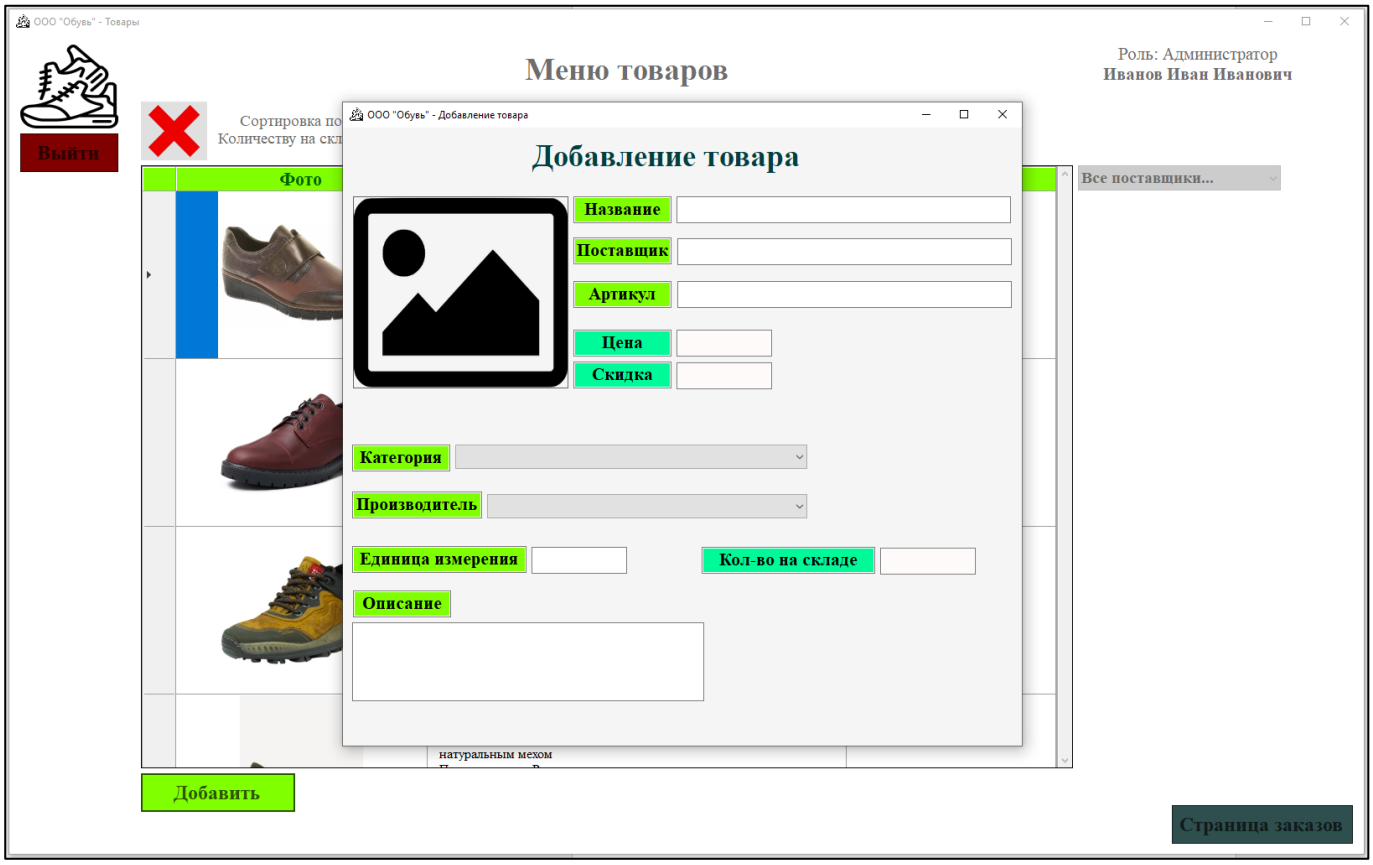

Рис. 9 – «Форма добавления товара: пустая»

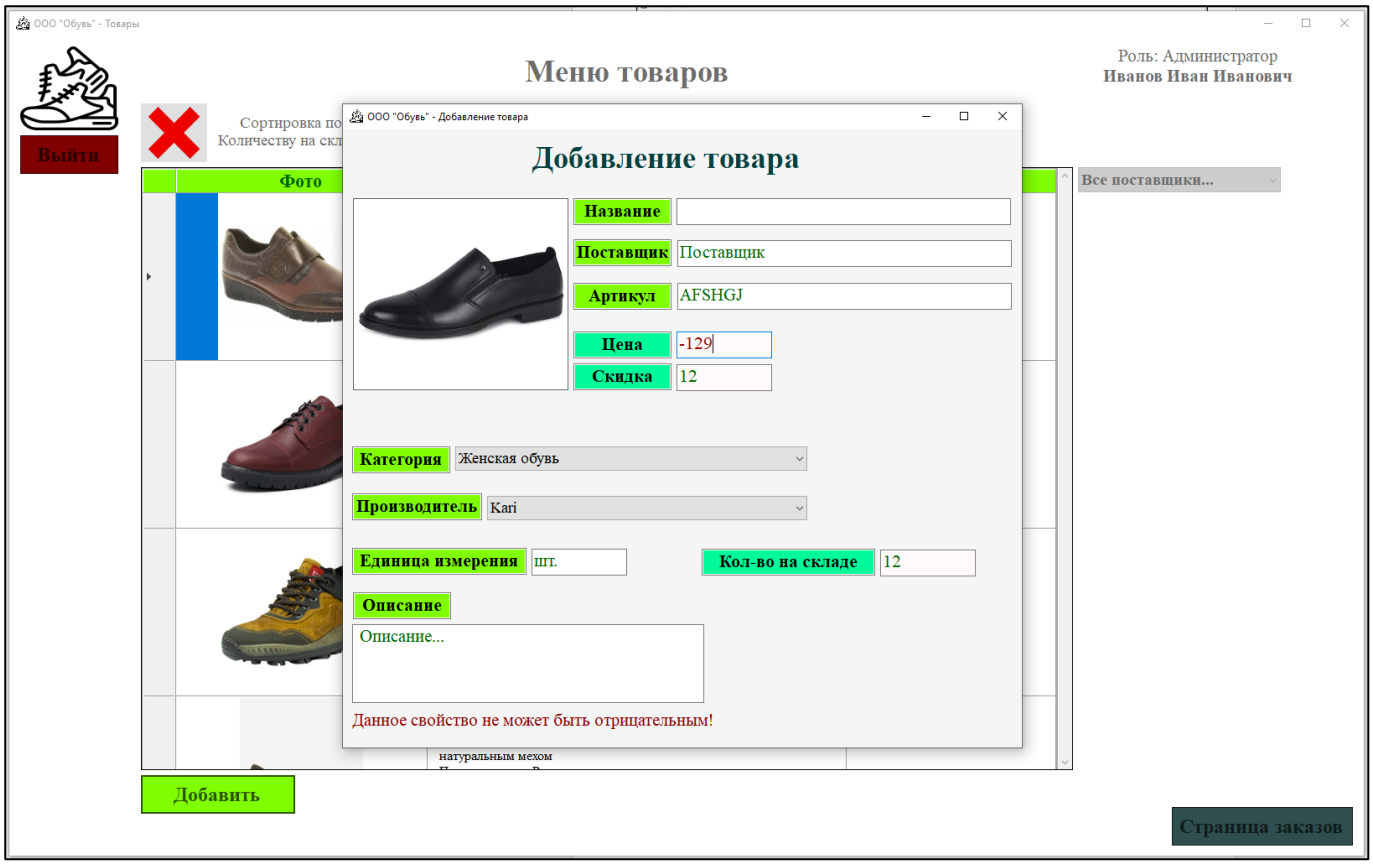

Рис. 10 – «Форма добавления товара: с ошибками»

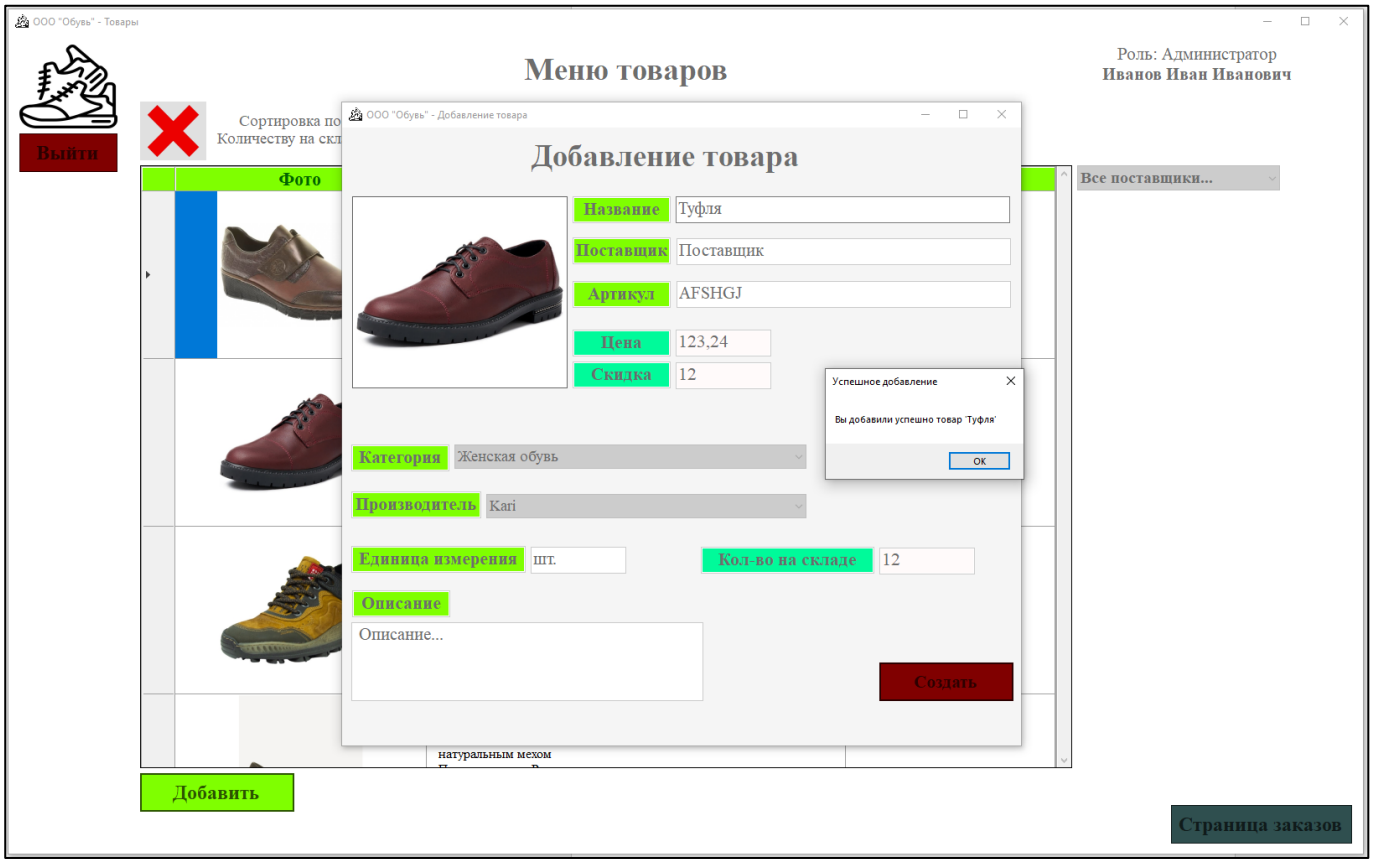

Рис. 11 – «Форма добавления товара: успешное добавление товара»

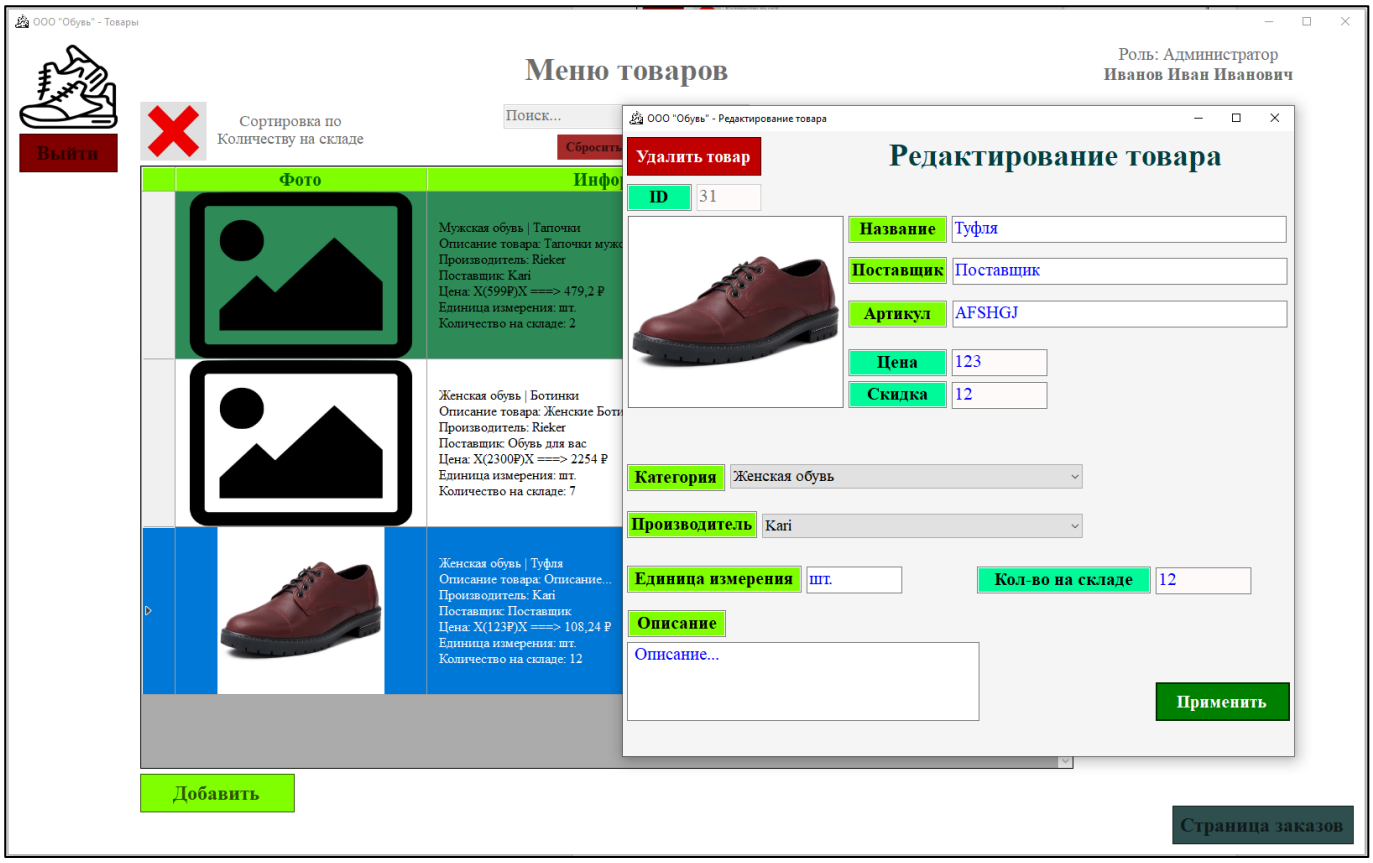

Рис. 12 – «Форма редактирования товара: открытие формы созданного товара»

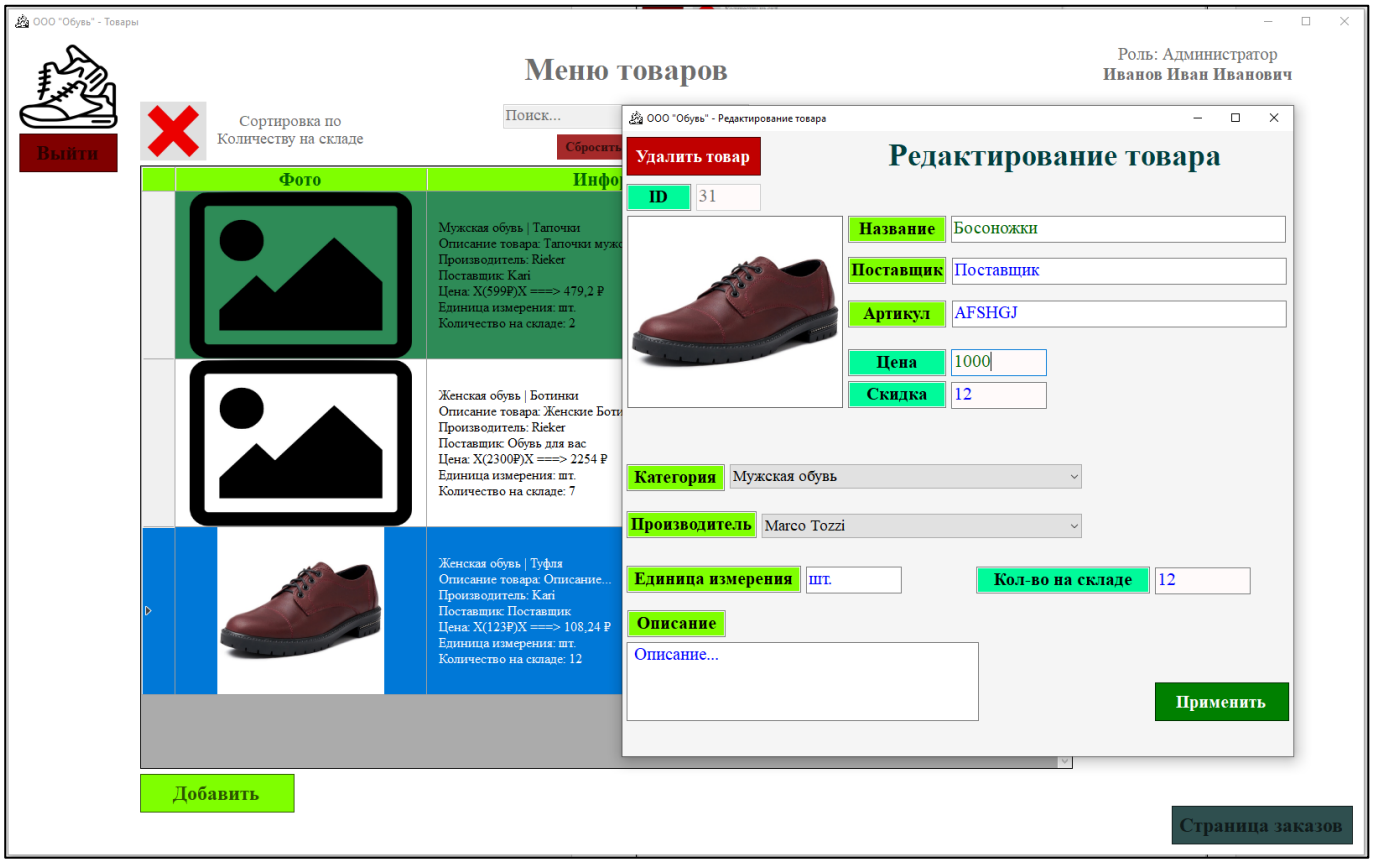

Рис. 13 – «Форма редактирования товара: редактирование товара»

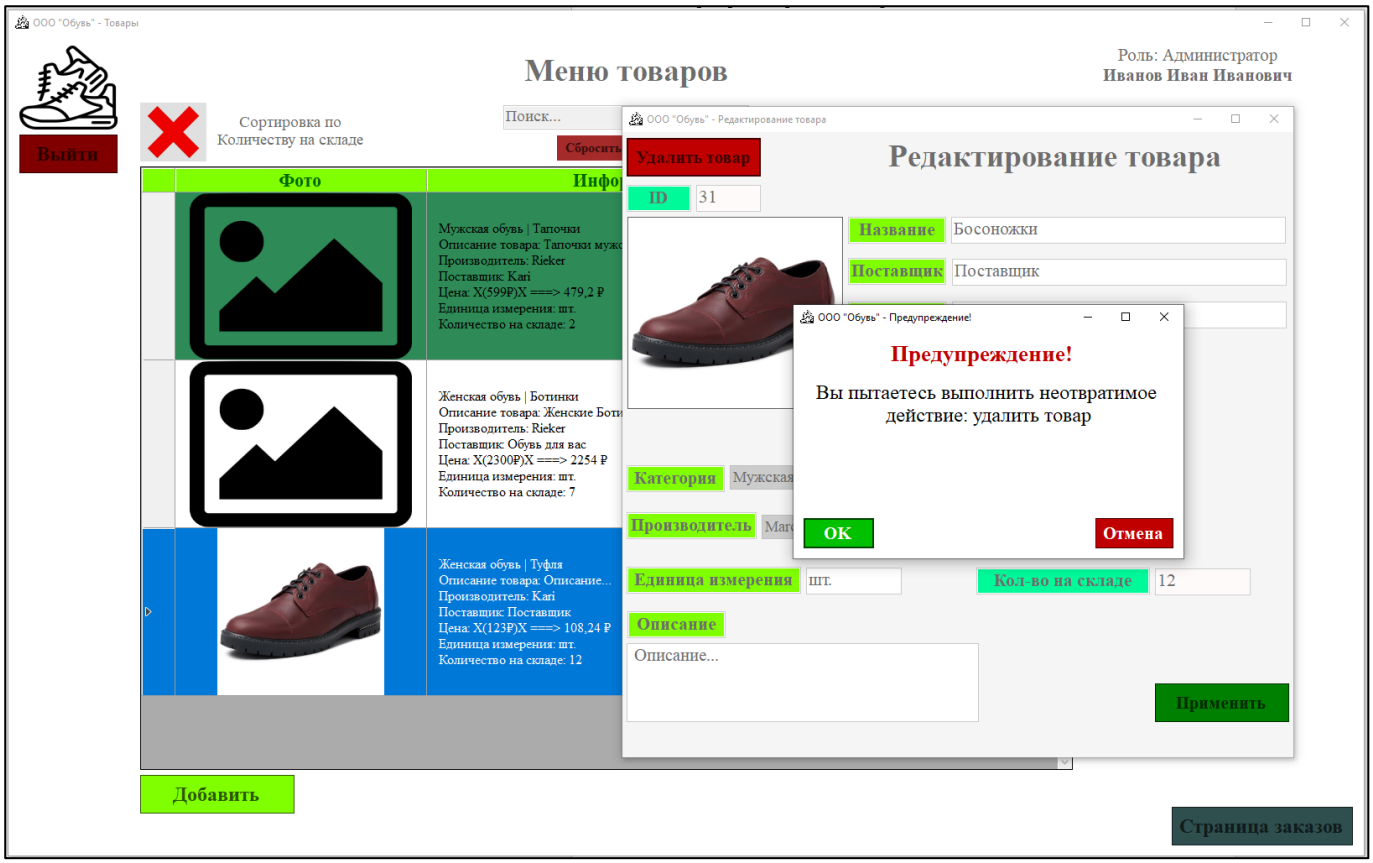

Рис. 14 – «Форма модального окна товара: открытие формы»

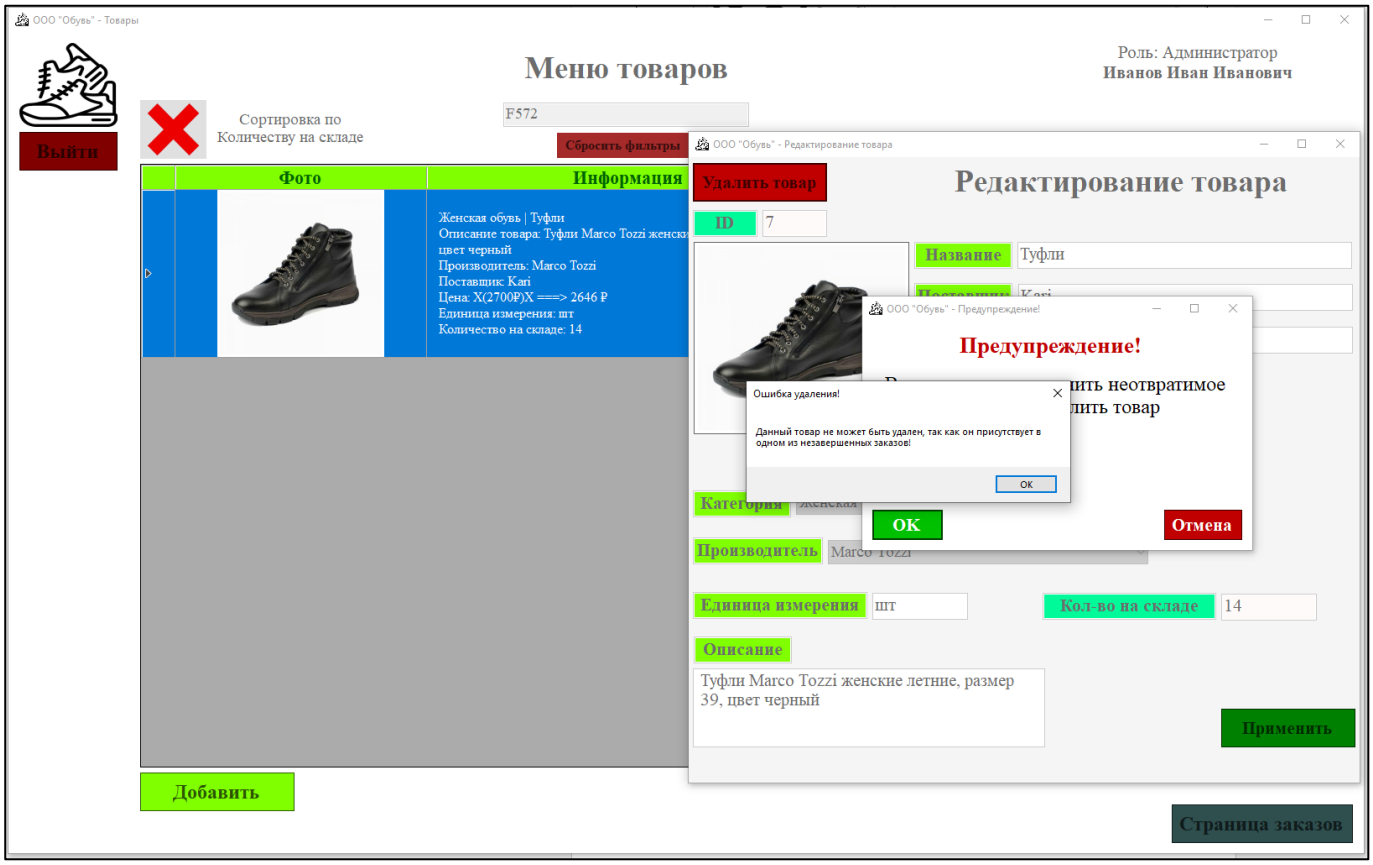

Рис. 15 – «Попытка удаления товара, присутствующего в незавершенном заказе»

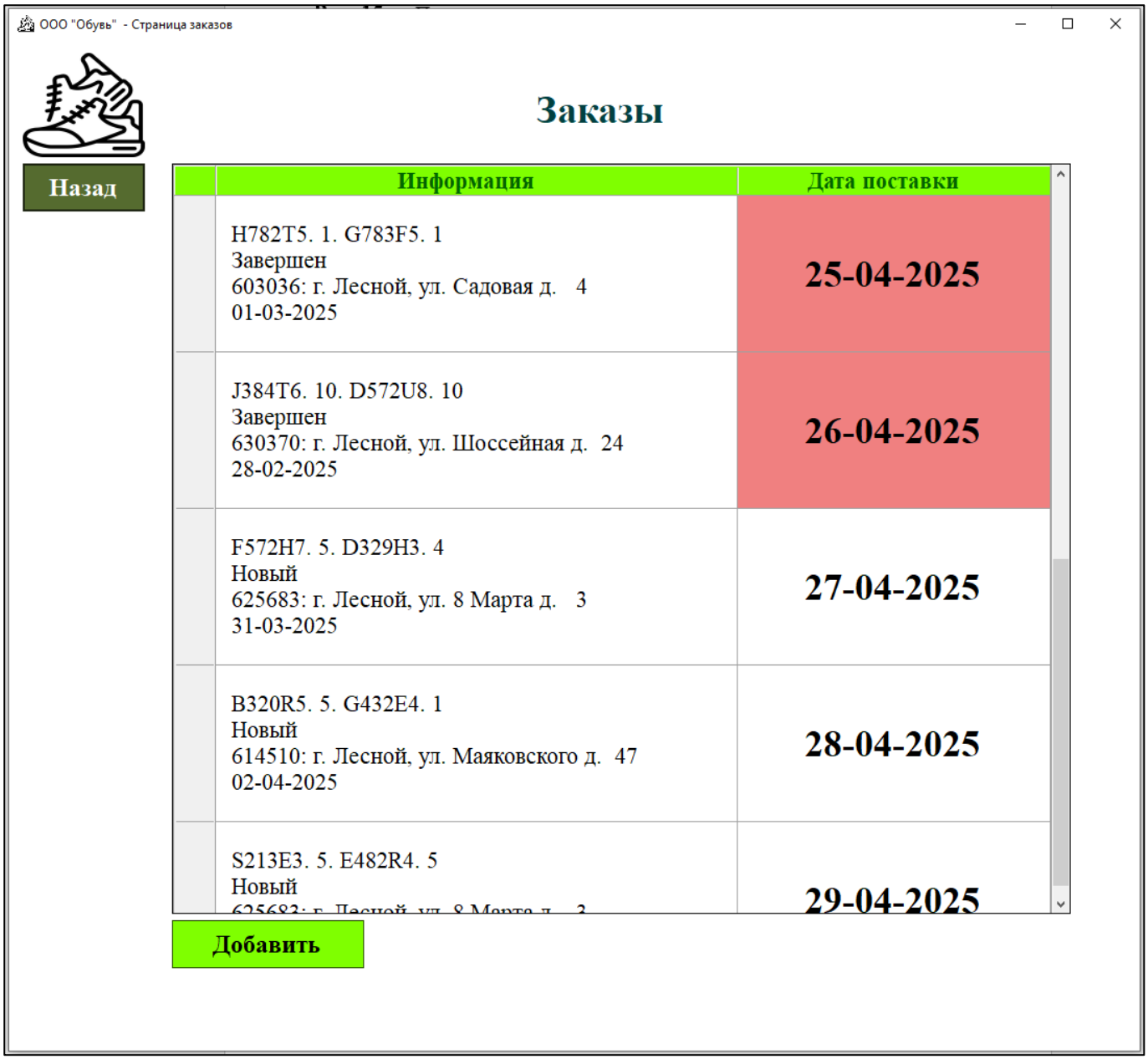

Рис. 16 – «Форма заказов: общий вид»

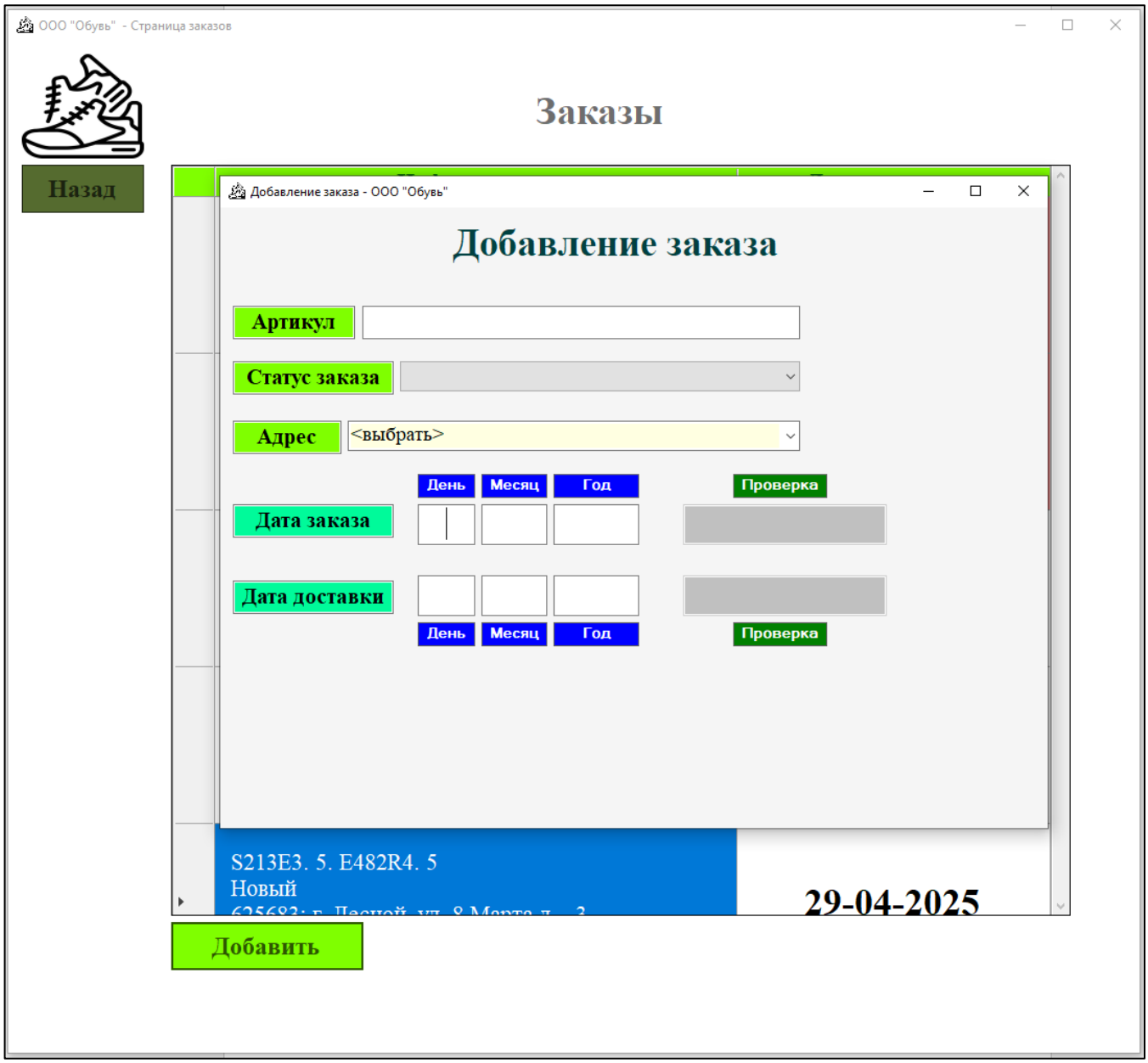

Рис. 17 – «Форма добавления заказа: пустая»

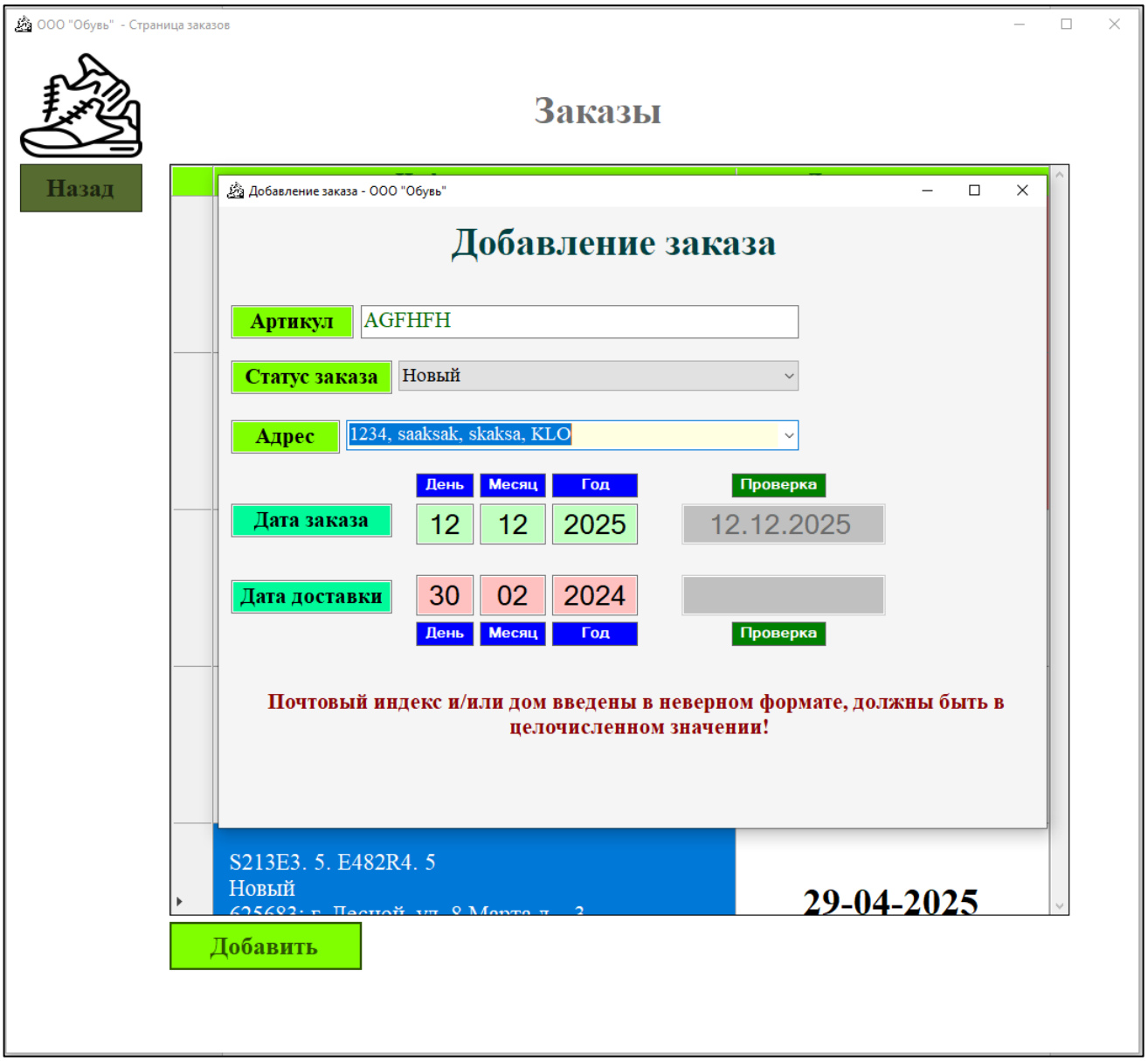

Рис. 18 – «Форма добавления заказа: с ошибками»

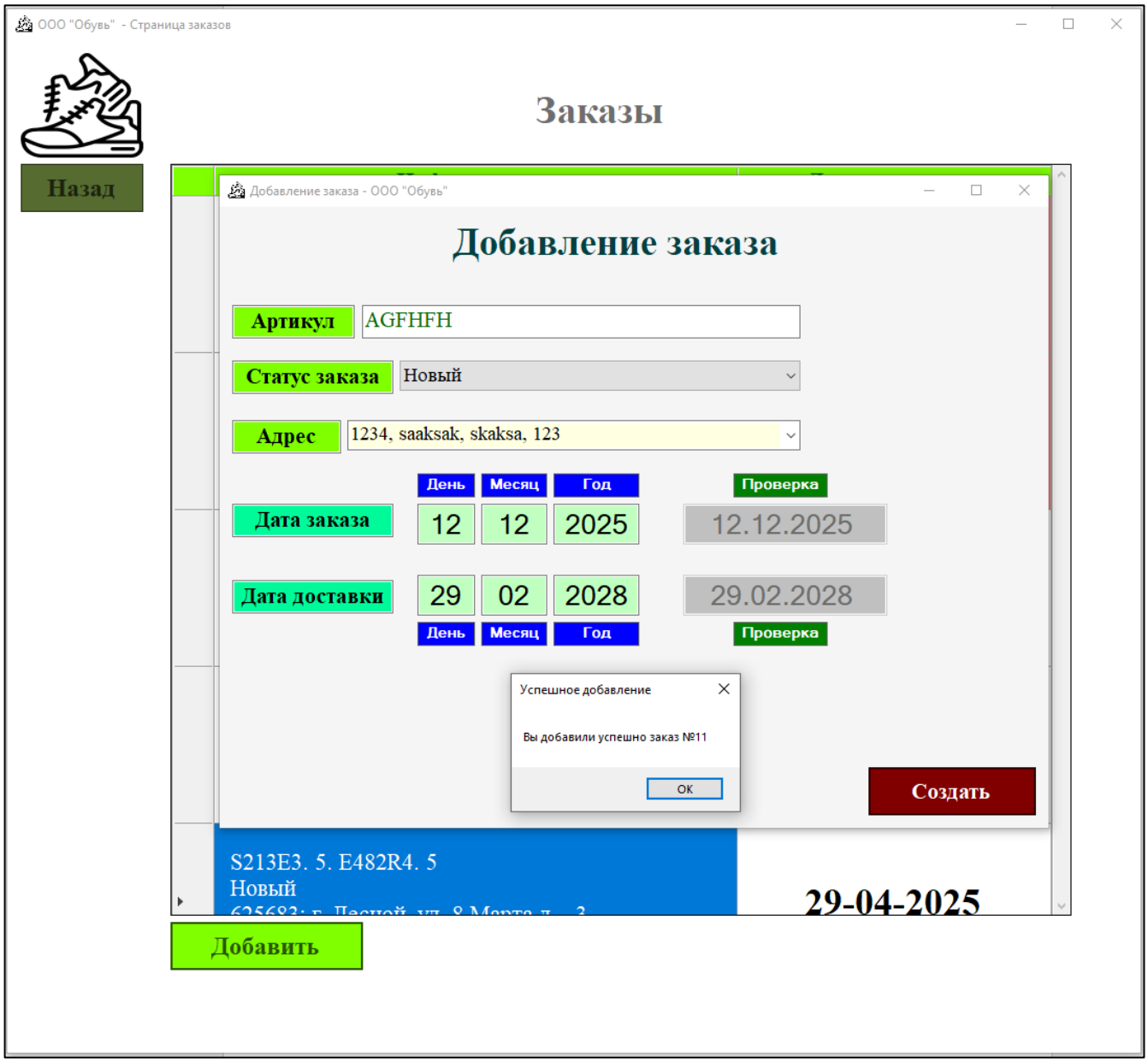

Рис. 19 – «Форма добавления заказа: успешное добавление»

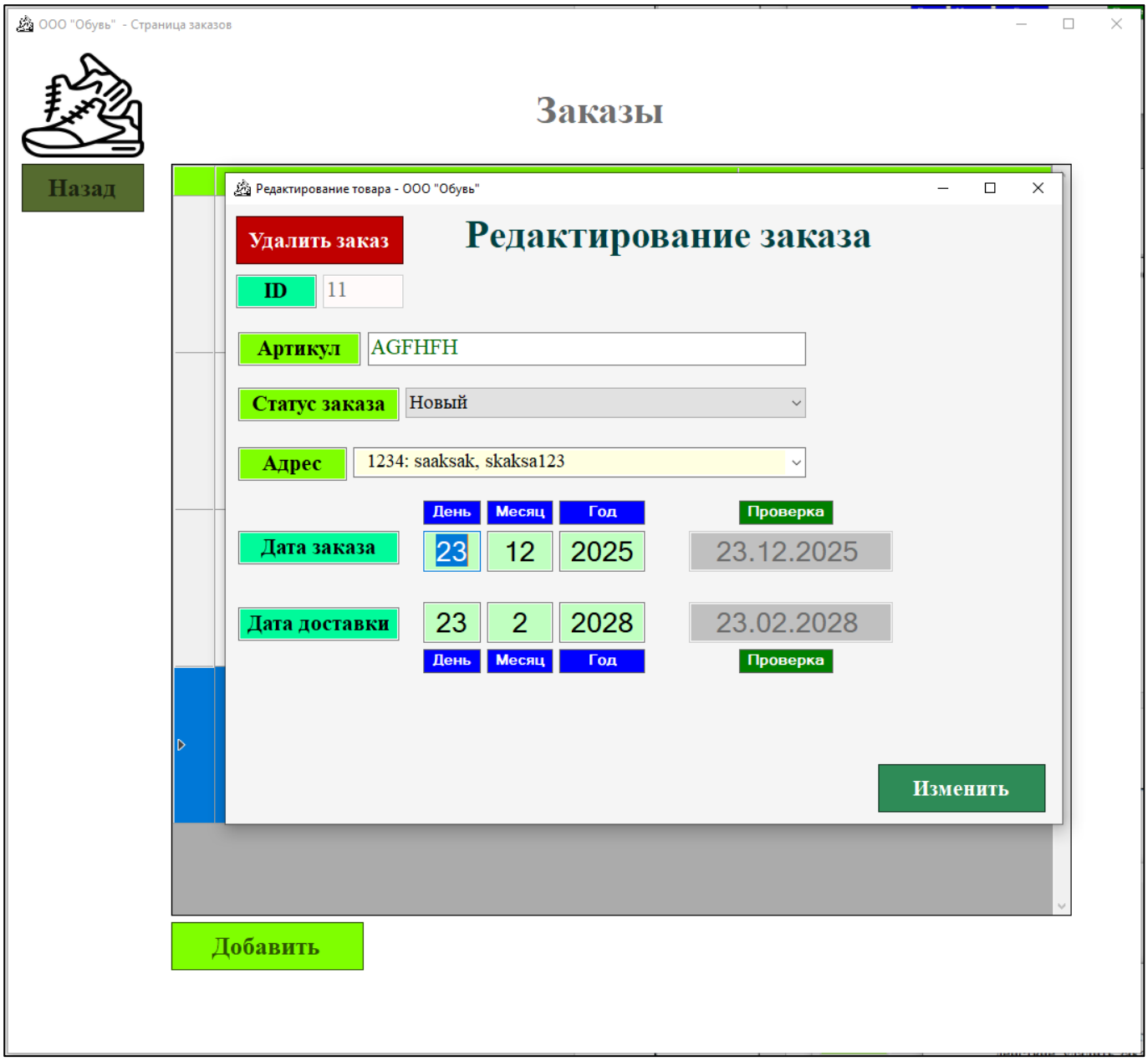

Рис. 20 – «Форма редактирования заказа: открытие формы»

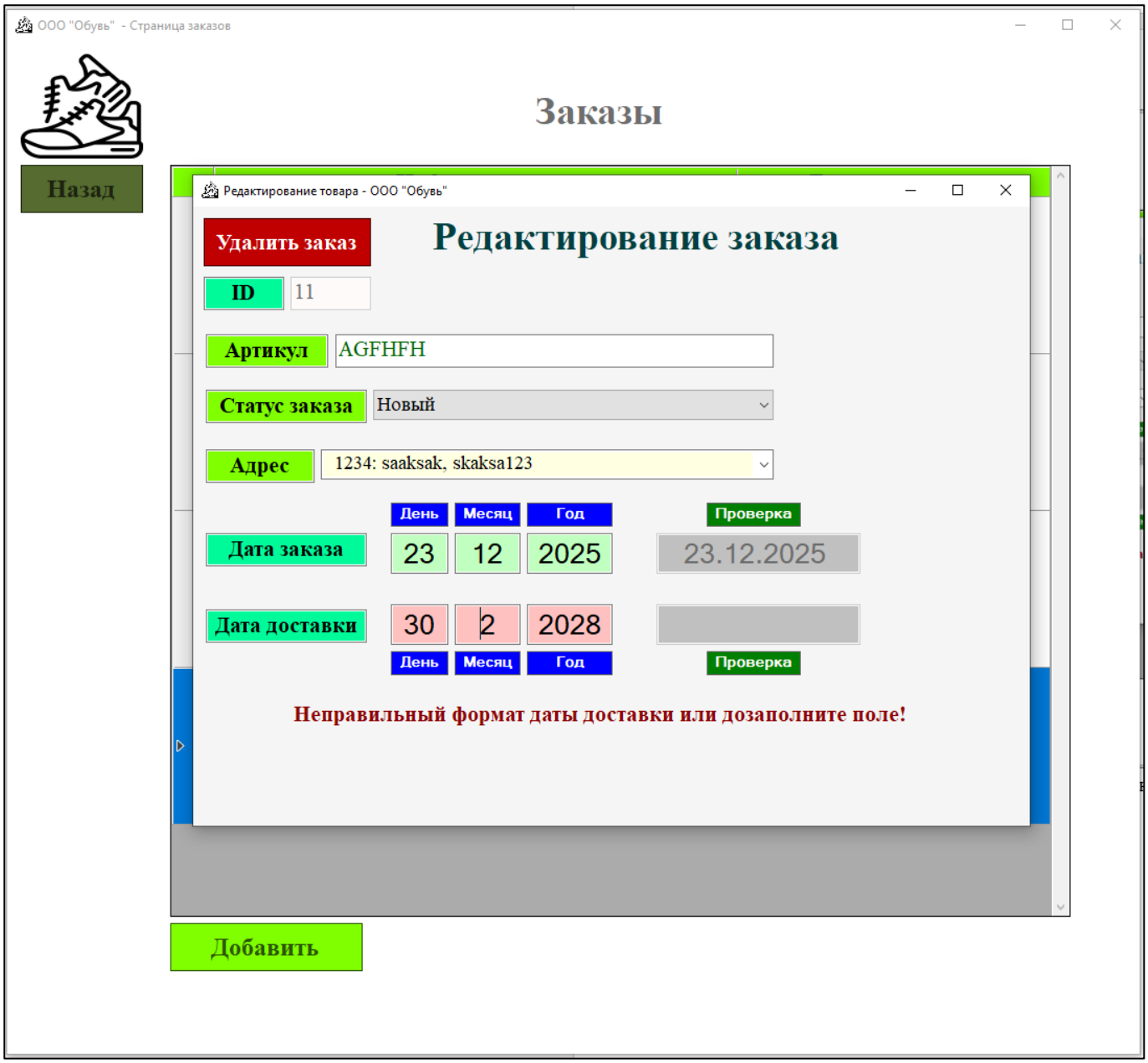

Рис. 21 – «Форма редактирования заказа: редактирование товара с ошибками»

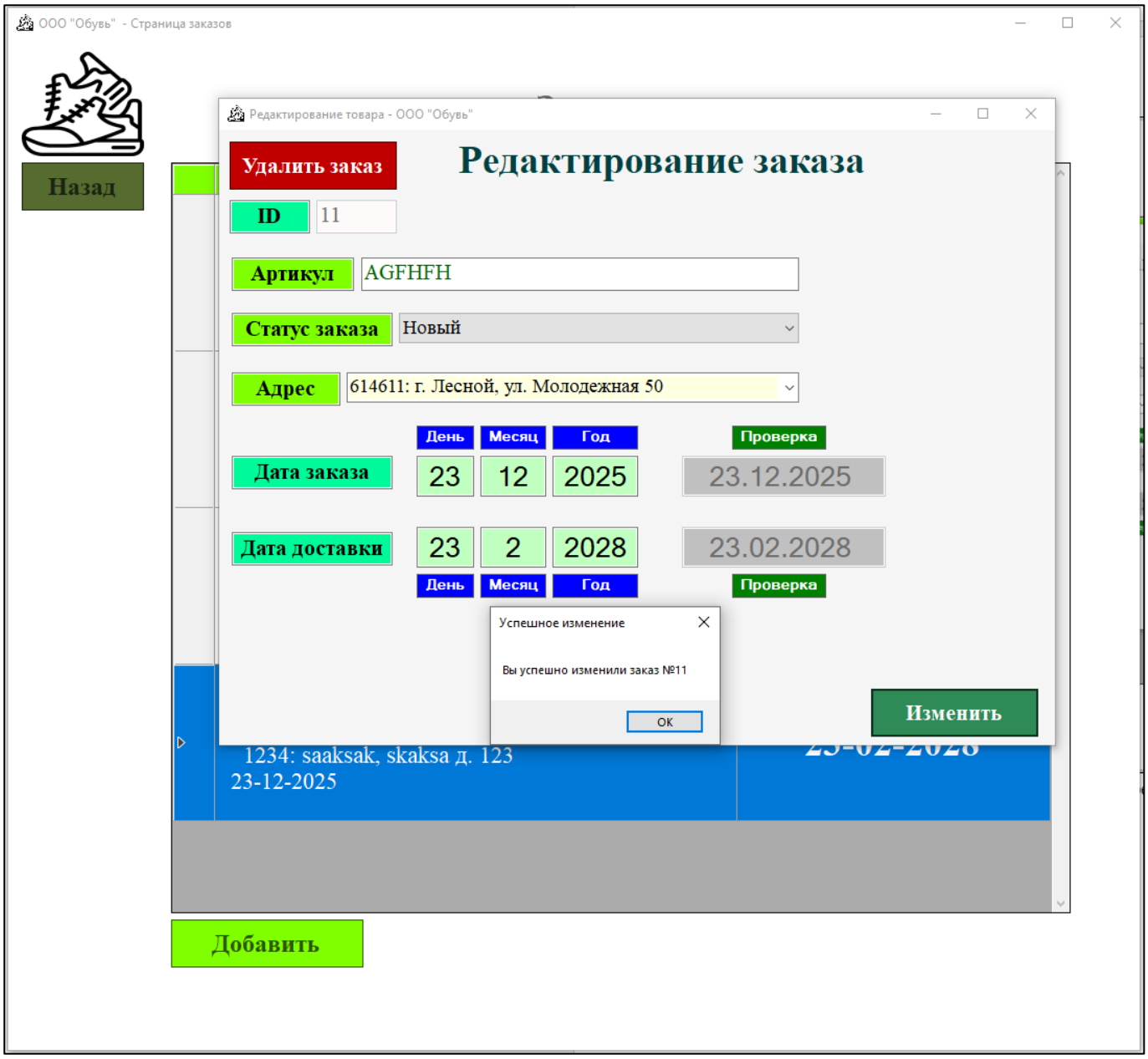

Рис. 22 – «Форма редактирования заказа: успешное редактирование товара»

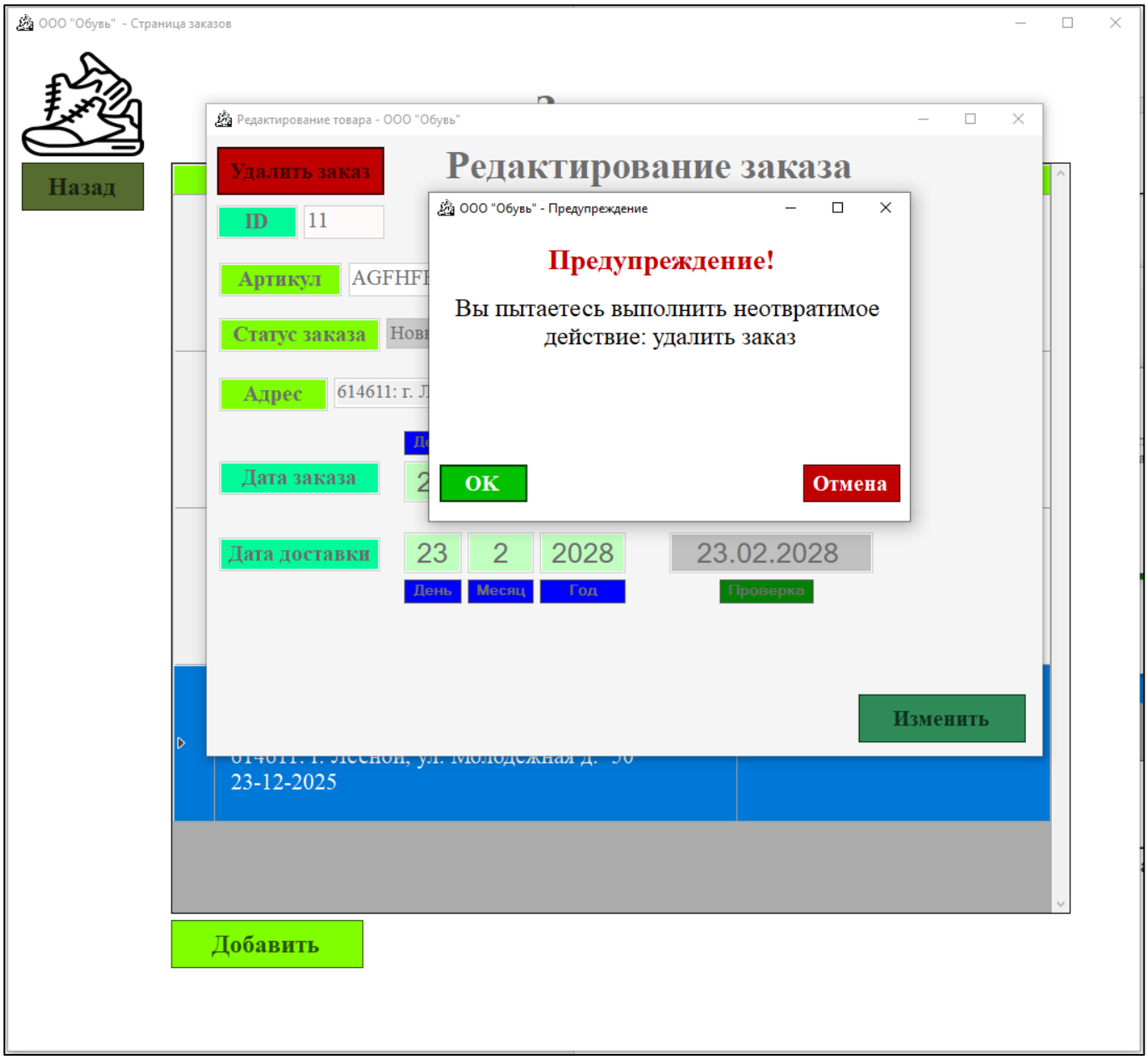

Рис. 23 – «Форма модального окна для заказа: общий вид»